

COMS BC1016

Introduction to Computational Thinking and Data Science

Lecture 3: Tables

Sept 30, 2025

Copyright © 2026 Barnard College

September 16, 2026

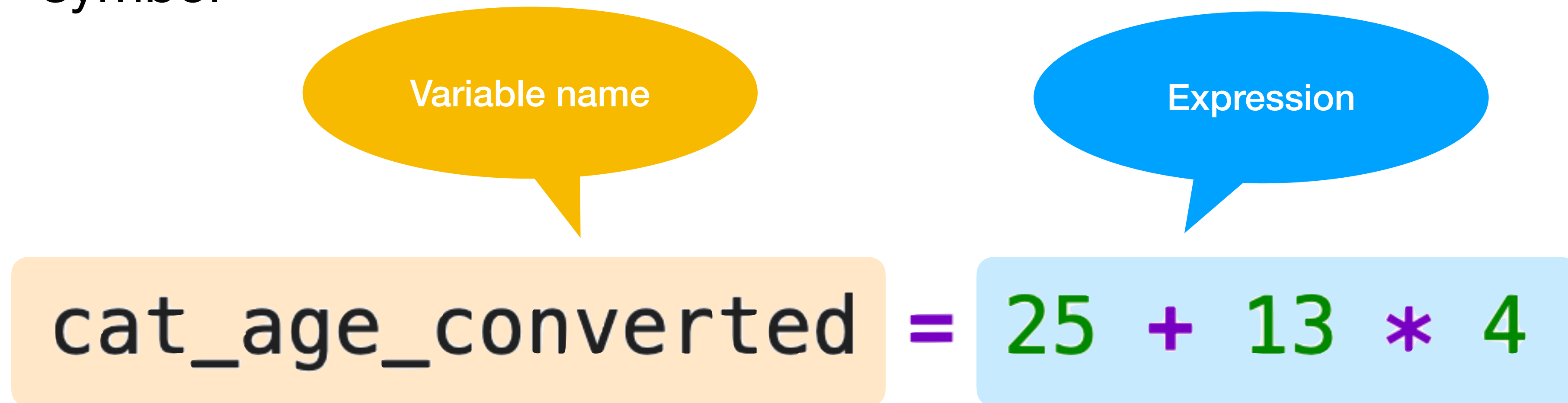
Lab Updates

- We were able to get a new room and increase the enrollment cap for the Section 1 lab! Please see new location:
 - Section 1: Wednesday 2:40-3:55 (LL001 Milstein)
 - Section 2: Wednesday 4:10-5:25 (516 Milstein)
 - Section 3: Thursday 9:40-11 (516 Milstein)
 - Section 4: Wednesday 4:10-5:25 (LL107 R&D Science Center)
- Labs start this week! Make sure you are registered for a lab!
- If you are on the wait-list, you should still attend and have the TA make a note that you were there

Last Time: Basics of Python

Variables and Assignments

- **Variables** are values you can assign values to
 - “Variable” because they can change
- An **assignment** statement (=) changes the value of the name to the left of the = symbol



Functions

- **Functions** are reusable blocks of code
 - To run a function, you type the function name following by parenthesis
 - Sometimes functions can take expressions as inputs (“**arguments**”)
 - Some useful functions aren’t built in, so we tell Python where to find them using **import**

```
import numpy  
numpy.sqrt(4)
```

```
2.0
```

Data Types

Type	Example
Int	3
Float	3.0
String	'three'
Functions	abs(-5)
Boolean	True
Arrays	[1, 2, 3, 4]

Last Time

Every Value has a Type

- The `type` function can tell you the type of a value
 - `type(2)`
 - `type(2 + 2)`
- An expression's type is based on its value, not how it looks
 - Example: What does this second line evaluate to?

```
two = 2
```

```
type(two)
```

Data Types (Continued)

Booleans

- True or False values
- Useful for conditional statements
 - Usually of the form “if some statement is true, then execute some code. Otherwise, do something else.”
- *We will talk more about Booleans in a few weeks*

Arrays

- **Arrays** are sequences of values

(Note: in this class we'll mostly use NumPy arrays)

- **Examples:**

- `[1, 2, 3, 5]`

- `["Mystery", "Abby", "Jinu", "Baby", "Romance"]`

- `['apples', 'bananas', 'apples', 'apples']`

- **Elements of an array should have the same type**

- **Can make arrays using** `datascience.make_array` **or** `numpy.array`

Arrays

Demo!

- Arrays are **zero-indexed**
 - The first element is the 0th and the last is length-1
- Arithmetic is applied to each element individually
- Adding two arrays adds the corresponding elements (but they must be the same length!)

```
from datascience import *  
onetwothree = make_array(1,2,3)  
onetwothree * 2
```

```
array([2, 4, 6])
```

```
from datascience import *  
onetwothree = make_array(1,2,3)  
twothreefour = make_array(2,3,4)  
onetwothree + twothreefour
```

```
array([3, 5, 7])
```

Arrays

- Can access the `i`th element either using `.item(i)` or `[i]`
 - Example (for an array `x`): `x.item(i)` or `x[i]`
- `len(x)` and `x.size` give the number of elements in the array `x`
- Remember: counting starts at 0 (sometimes referred to as being 0-indexed)
 - `x[0]` gives the first element
 - `x[len(x) - 1]` gives the last element

Arrays

- Many useful functions for operating on arrays
 - Helpful to be aware of, but you do not need to memorize them!

Function	Description
<code>np.prod</code>	Multiply all elements together
<code>np.sum</code>	Add all elements together
<code>np.all</code>	Test whether all elements are true values (non-zero numbers are true)
<code>np.any</code>	Test whether any elements are true values (non-zero numbers are true)
<code>np.count_nonzero</code>	Count the number of non-zero elements

Function	Description
<code>np.char.lower</code>	Lowercase each element
<code>np.char.upper</code>	Uppercase each element
<code>np.char.strip</code>	Remove spaces at the beginning or end of each element
<code>np.char.isalpha</code>	Whether each element is only letters (no numbers or symbols)
<code>np.char.isnumeric</code>	Whether each element is only numeric (no letters)

Function	Description
<code>np.diff</code>	Difference between adjacent elements
<code>np.round</code>	Round each number to the nearest integer (whole number)
<code>np.cumprod</code>	A cumulative product: for each element, multiply all elements so far
<code>np.cumsum</code>	A cumulative sum: for each element, add all elements so far
<code>np.exp</code>	Exponentiate each element
<code>np.log</code>	Take the natural logarithm of each element
<code>np.sqrt</code>	Take the square root of each element
<code>np.sort</code>	Sort the elements

Function	Description
<code>np.char.count</code>	Count the number of times a search string appears among the elements of an array
<code>np.char.find</code>	The position within each element that a search string is found first
<code>np.char.rfind</code>	The position within each element that a search string is found last
<code>np.char.startswith</code>	Whether each element starts with the search string

Ranges

Demo!

A **range** is an array of **consecutive numbers**:

- `np.arange(end)`

Create an array of increasing integers from 0 up to `end`

- `np.arange(start, end)`

Create an array of increasing integers from `start` up to `end`

- `np.arange(start, end, step)`

A range where `step` is added between consecutive values

The range always includes `start` but *excludes* `end`

Tables

Tables

A **table** is a way of representing data sets

- Each **row** is an **individual**
- Each **column** is an **attribute** of the individual

Name	Age	Coloring	Favorite Food
Gertrude	15 yrs	Tuxedo	Milk
Ruby	14 yrs	Tuxedo	Potato chips
Corina	6 yrs	Dilute Tortoiseshell	Kibble
Frito	1 yr	Tabby	Cheese

Table Columns are Arrays

Demo!

- A column of a table is an array of values (one per row)
- For a table `t`, use `t.column(label)` or `t.column(index)` to get the array
- Can get individual values or perform computations on the array

Name
Gertrude
Ruby
Corina
Frito

Age
15 yrs
14 yrs
6 yrs
1 yr

Coloring
Tuxedo
Tuxedo
Dilute Tortoiseshell
Tabby

Favorite Food
Milk
Potato chips
Kibble
Cheese

Ways to create a table

- `Table.read_table(filename)` reads a table from a file (such as a spreadsheet or CSV)
- `Table()` creates an empty table to which columns can be added
 - `.with_column(label, values)` and `.with_columns(label1, value1, label2, value2,...)` adds columns onto the end of a table

Ways to create a table

- `Table.read_table(filename)` reads a table from a file (such as a spreadsheet or CSV)
- `Table()` creates an empty table to which columns can be added
 - `.with_column(label, values)` and `.with_columns(label1, value1, label2, value2,...)` adds columns onto the end of a table
- **Example:** `Table().with_columns("Name",
make_array("Gertrude", "Ruby", "Corina", "Frito"))`

Ways to create a table

- `Table(filename)` reads a table from a file (such as a spreadsheet)
- `Table()` creates an empty table
- `.with_columns()` adds a column
- `.with_column(label, values)` and `.with_column(label, value1, label2, value2,...)` adds columns onto the end of a table
- Example: `Table().with_columns("Name", make_array("Gertrude", "Ruby", "Corina", "Frito"))`

`Table()` creates an empty table

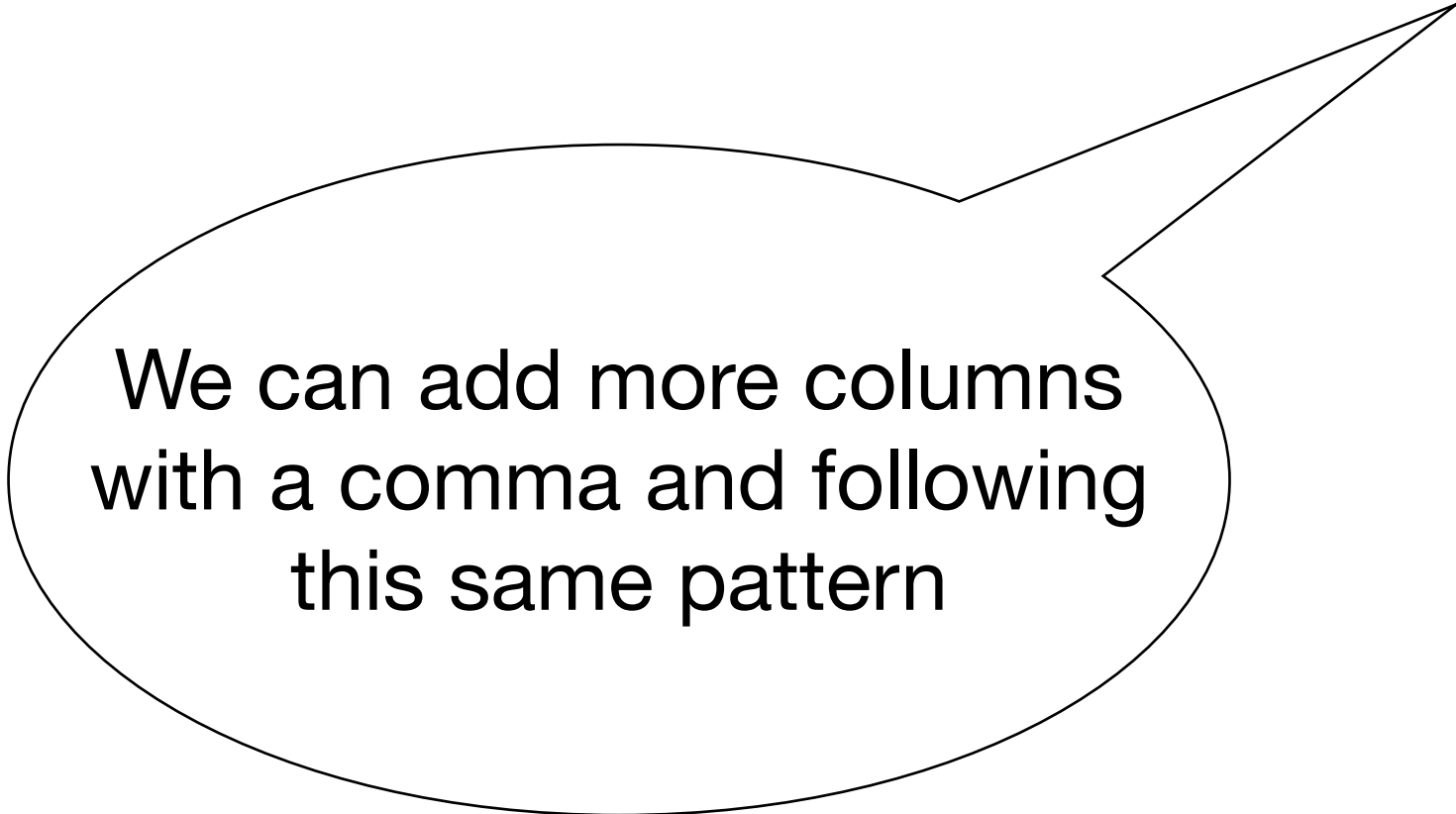
`.with_columns()` adds a column

The first argument to `.with_columns` is the column name

... Followed by an array with the column values

Ways to create a table

- `Table.read_table(filename)` reads a table from a file (such as a spreadsheet or CSV)
- `Table()` creates an empty table to which columns can be added
 - `.with_column(label, values)` and `.with_columns(label1, value1, label2, value2,...)` adds columns onto the end of a table
- **Example:** `Table().with_columns("Name",
make_array("Gertrude", "Ruby", "Corina", "Frito"),
"Age", make_array(15, 14, 6, 1))`



We can add more columns with a comma and following this same pattern

Ways to create a table

Demo!

- `Table.read_table(filename)` reads a table from a file (such as a spreadsheet or CSV)
- `Table()` creates an empty table to which columns can be added
- `.with_column(label, values)` and `.with_columns(label1, value1, label2, value2,...)` adds columns onto the end of a table
- **Example:** `Table().with_columns("Name",
make_array("Gertrude", "Ruby", "Corina", "Frito"),
"Age", make_array(15, 14, 6, 1))`

Name	Age
Gertrude	15
Ruby	14
Corina	6
Frito	1

More Ways to Create Tables

Demo!

- Create a new table from an existing table `tbl` with columns `c`, `c1`, `c2`

- `tbl.select(c1, c2, ...)`

Table with only columns `c1`, `c2`, ...

- `tbl.drop(c1, c2, ...)`

Table without columns `c1`, `c2`, ...

- `tbl.sort(c[, descending=False])`

Table sorted by elements in column `c`

- `tbl.where(c, predicate)`

Only rows in the table where the value in column `c` satisfies the predicate

- `tbl.take(row_indices)`

only the specified rows

Filtering

<https://www.data8.org/sp22/python-reference.html>

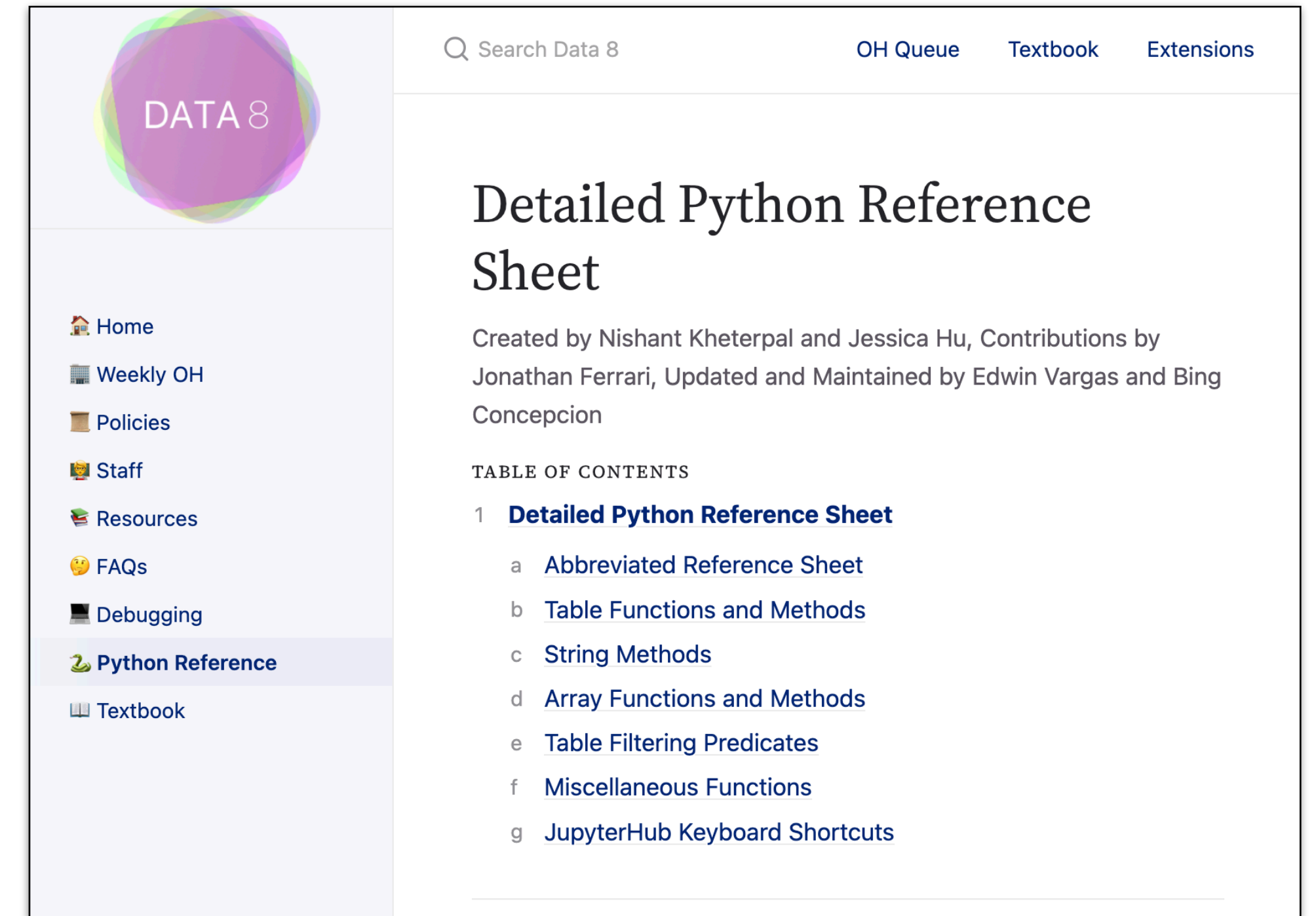
Table.where Predicates

Any of these predicates can be negated by adding `not_` in front of them, e.g. `are.not_equal_to(Z)` or `are.not_containing(S)`.

Predicate	Description
<code>are.equal_to(Z)</code>	Equal to <code>Z</code>
<code>are.not_equal_to(Z)</code>	Not equal to <code>Z</code>
<code>are.above(x)</code>	Greater than <code>x</code>
<code>are.above_or_equal_to(x)</code>	Greater than or equal to <code>x</code>
<code>are.below(x)</code>	Less than <code>x</code>
<code>are.below_or_equal_to(x)</code>	Less than or equal to <code>x</code>
<code>are.between(x,y)</code>	Greater than or equal to <code>x</code> and less than <code>y</code>
<code>are.between_or_equal_to(x,y)</code>	Greater than or equal to <code>x</code> , and less than or equal to <code>y</code>
<code>are.contained_in(A)</code>	Is a substring of <code>A</code> (if <code>A</code> is a string) or an element of <code>A</code> (if <code>A</code> is a list/array)
<code>are.containing(S)</code>	Contains the string <code>S</code>
<code>are.strictly_between(x,y)</code>	Greater than <code>x</code> and less than <code>y</code>

Python Reference

<https://www.data8.org/sp25/reference/>



The screenshot shows the Data 8 website's Python Reference page. The header includes a search bar, "OH Queue", "Textbook", and "Extensions" links. A sidebar on the left contains links to Home, Weekly OH, Policies, Staff, Resources, FAQs, Debugging, Python Reference (highlighted), and Textbook. The main content area is titled "Detailed Python Reference Sheet" and credits Nishant Kheterpal, Jessica Hu, Jonathan Ferrari, Edwin Vargas, and Bing Concepcion. It includes a "TABLE OF CONTENTS" with links to an abbreviated reference sheet, table functions, string methods, array functions, table filtering predicates, miscellaneous functions, and JupyterHub keyboard shortcuts.

DATA 8

Search Data 8 OH Queue Textbook Extensions

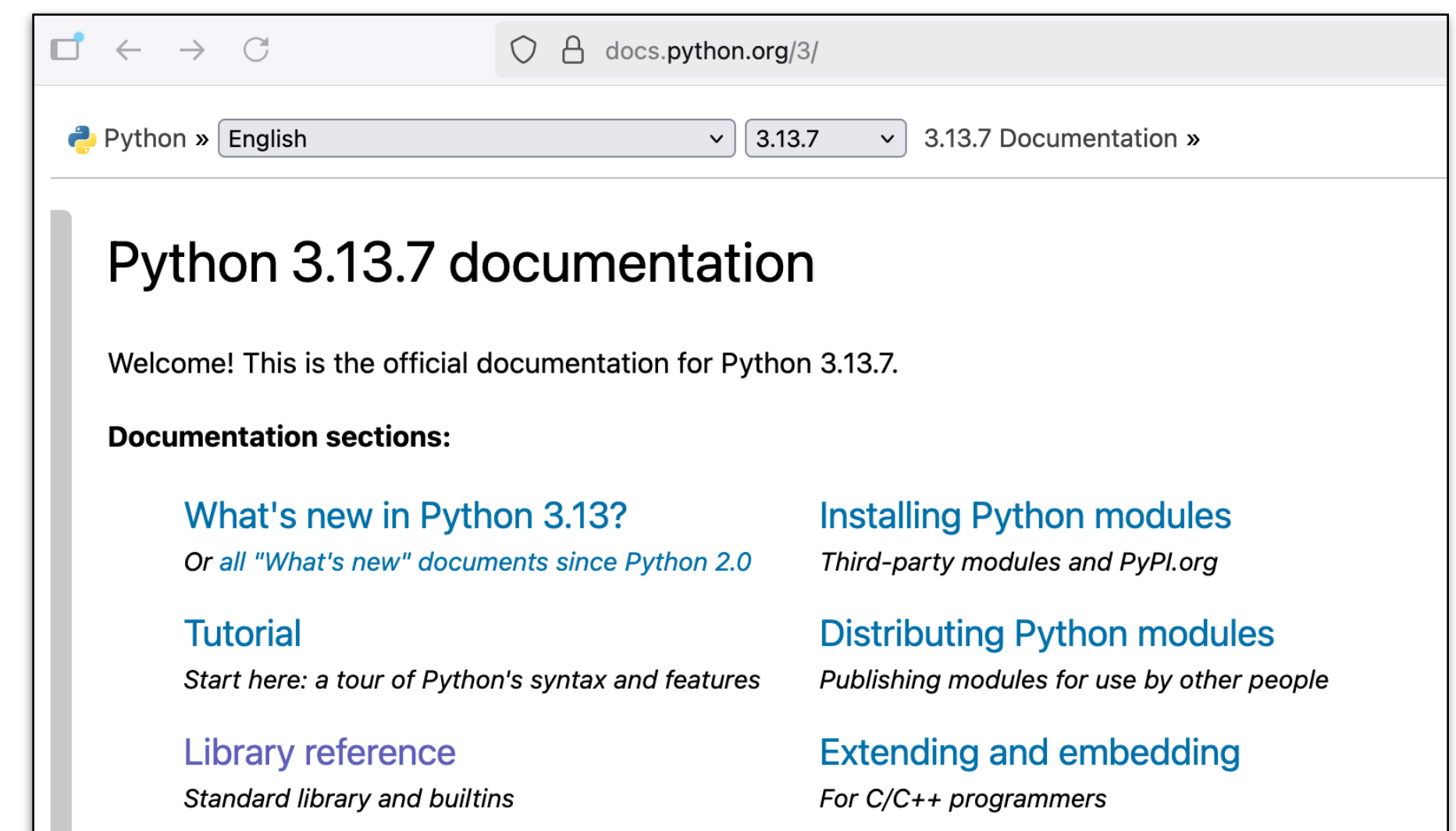
Detailed Python Reference Sheet

Created by Nishant Kheterpal and Jessica Hu, Contributions by Jonathan Ferrari, Updated and Maintained by Edwin Vargas and Bing Concepcion

TABLE OF CONTENTS

- 1 **Detailed Python Reference Sheet**
 - a [Abbreviated Reference Sheet](#)
 - b [Table Functions and Methods](#)
 - c [String Methods](#)
 - d [Array Functions and Methods](#)
 - e [Table Filtering Predicates](#)
 - f [Miscellaneous Functions](#)
 - g [JupyterHub Keyboard Shortcuts](#)

<https://docs.python.org/3/>



The screenshot shows the official Python 3.13.7 documentation website. The browser address bar shows "docs.python.org/3/". The page has a header with "Python »", a language dropdown set to "English", a version dropdown set to "3.13.7", and a link to "3.13.7 Documentation ». The main heading is "Python 3.13.7 documentation". Below it, a welcome message states: "Welcome! This is the official documentation for Python 3.13.7." A section titled "Documentation sections:" lists several links: "What's new in Python 3.13?" (with a sub-link for "What's new" documents since Python 2.0), "Installing Python modules" (with a sub-link for third-party modules and PyPI.org), "Tutorial" (with a sub-link for a tour of Python's syntax and features), "Distributing Python modules" (with a sub-link for publishing modules for use by other people), "Library reference" (with a sub-link for standard library and builtins), and "Extending and embedding" (with a sub-link for C/C++ programmers).

docs.python.org/3/

Python » English 3.13.7 3.13.7 Documentation »

Python 3.13.7 documentation

Welcome! This is the official documentation for Python 3.13.7.

Documentation sections:

- [What's new in Python 3.13?](#)
Or all "What's new" documents since Python 2.0
- [Installing Python modules](#)
Third-party modules and PyPI.org
- [Tutorial](#)
Start here: a tour of Python's syntax and features
- [Distributing Python modules](#)
Publishing modules for use by other people
- [Library reference](#)
Standard library and builtins
- [Extending and embedding](#)
For C/C++ programmers

Next Class

- Today
 - Jupyter Notebooks
 - Data Types
- Wednesday
 - Tables
- Monday
 - Charts & Visualization